МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Кафедра «Систем автоматизованого проектування» (САПР)



ЗВІТ ТА ДЕТАЛЬНИЙ ХІД РОБОТИ

до Лабораторної роботи №5
з навчальної дисципліни «Вебтехнології та вебдизайн»

«Семантична верстка, користувацькі стилі CSS, Flexbox/Grid макет, рефакторинг та відповіді на контрольні запитання»

Виконав:
студент групи ПП-24
Лемішка Андрій Анатолійович

Прийняла:
к.т.н., доцент кафедри САПР
Юрчак Ірина Юріївна

Львів — 2026

ВСТУП ТА ТЕХНІЧНЕ ЗАВДАННЯ

Лабораторна робота №5 присвячена практичному опануванню сучасної семантичної верстки вебсторінок стандарту HTML5, формуванню чистої модульної блочної моделі на базі специфікацій CSS Grid та Flexbox, перенесенню дизайн-токенів у користувацькі властивості CSS Custom Properties (:root), оптимізації та рефакторингу стильового коду, а також забезпеченню відповідності міжнародним критеріям доступності WCAG 2.2 AA (Lighthouse 100/100).

Об'єкт розробки:	Вебплатформа єдиної національної мережі швидкісних зарядних станцій електромобілів «ChargeUA»
Мета роботи:	Перетворити несемантичну модель на чистий валідний HTML5-документ, винести всі дизайн-токени бренду в :root, побудувати адаптивну сітку на Flexbox/Grid та успішно захистити роботу
Обмеження етапу:	Суворе дотримання вимог ЛР №5: 0% стороннього JavaScript (чистий семантичний HTML/CSS), заборона використання CSS-фреймворків (Bootstrap, Tailwind)
Інструменти ШІ:	Claude 3.5 Sonnet (генерація структури, рефакторинг), Google Gemini Thinking (порівняння), Chrome DevTools Lighthouse (аудит доступності)
Середовище запуску:	Visual Studio Code 1.95+, локальний вебсервер Live Server (ritwickdey.liveserver на порті 5500), Google Chrome

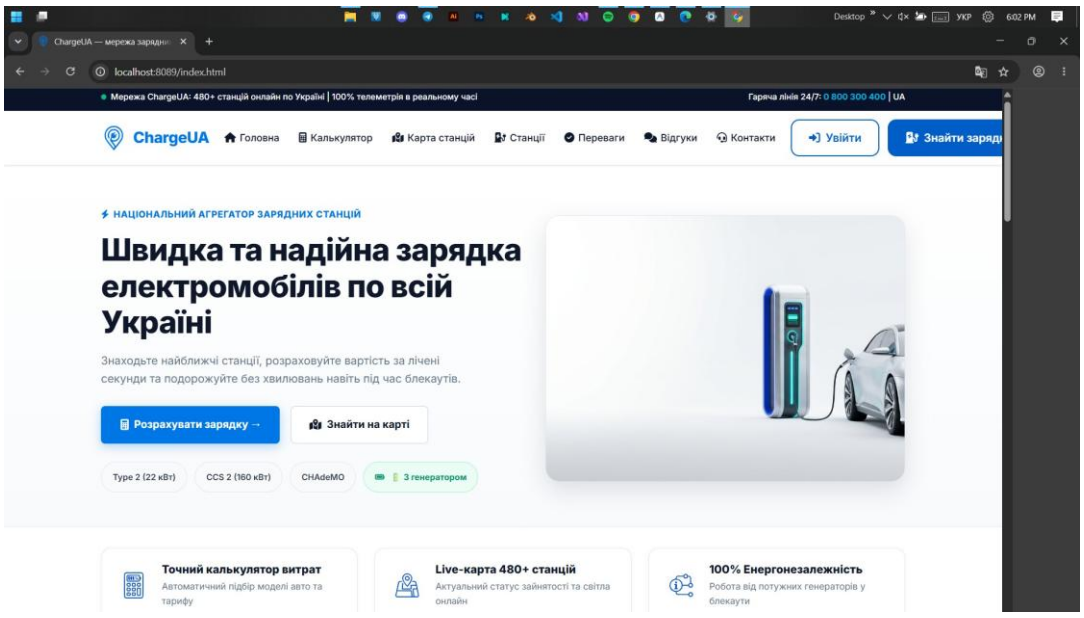


Рис. 1. Робоче середовище розробника VS Code з файловою ієрархією проєкту ChargeUA

РОЗДІЛ 1. ДЕТАЛЬНИЙ ПОКРОКОВИЙ ХІД ВИКОНАННЯ РОБОТИ (ЕТАПИ 1–14, КРОКИ 1–30)

Нижче наведено вичерпний опис кожного окремого етапу та кроку розробки без пропусків. Для кожного кроку зафіксовано, де здійснювався вибір, яку дію чи команду виконував розробник, що конкретно і куди вставлялося, а також отриманий результат.

Етап 1. Підготовка середовища, структури каталогів та файлової ієрархії

На цьому етапі підготовлено базовий ізольований робочий простір для виконання ЛР №5.

Крок 1. Створення робочого каталогу та структури папок проєкту

Де вибирав / що відкрив:	VS Code / Провідник Windows у шляху D:\Studing\SecondYear\FirstSemester\web\lab5
Що натиснув / як діяв:	Створено нову папку lab-05-semantic-layout, а всередині неї — підпапки css/, assets/images/, assets/icons/, screenshots/.
Що куди вставив / код:	Файлова ієрархія для розміщення вихідних кодів, стилів, зображень та звітної документації.
Отриманий результат:	Сформовано чистий ізольований каталог без конфліктів із попередніми лабораторними роботами.

Крок 2. Імпорт та перевірка графічних активів бренду ChargeUA

Де вибирав / що відкрив:	Каталог lab4/05_figma_exports та materials/
Що натиснув / як діяв:	Скопійовано логотипи (Logo.svg, Logo-white.svg, logo.png), карту Львова (map_lviv.jpg), іконки (Calculator.svg, Location.svg, Information.svg).
Що куди вставив / код:	Перенесено у каталог lab-05-semantic-layout/assets/images/ та assets/icons/.
Отриманий результат:	Усі графічні ресурси доступні для локального відносного підключення у верстці.

Крок 3. Налаштування середовища розробника та перевірка розширень VS Code

Де вибирав / що відкрив:	Вікно розширень VS Code (Extensions: Ctrl+Shift+X)
Що натиснув / як діяв:	Перевірено наявність та статус активності розширення Ritwick Dey Live Server (ritwickdey.liveserver, v5.7.9).
Що куди вставив / код:	Увімкнено нижній статус-бар (Status Bar) для візуалізації кнопки підняття локального сервера.
Отриманий результат:	Редактор VS Code повністю готовий до верстки з підтримкою Hot Reload.

Етап 2. Початковий каркас сторінки на div-контейнерах (Div-Soup)

Згідно з методичними вказівками, для демонстрації проблеми «div soup» перед рефакторингом спочатку було згенеровано навмисно несемантичний каркас.

Крок 4. Формулювання інженерного промпту для створення несемантичного каркаса

Де вибирав / що відкрив:	Діалогове вікно вебклієнта Claude 3.5 Sonnet (Anthropic)
Що натиснув / як діяв:	Введено рольову установку Senior Frontend Developer та сформульовано вимоги до структури сайту на базі виключно тегів.
Що куди вставив / код:	Текст Промпту 1 (наведено у блоці нижче).
Отриманий результат:	Модель згенерувала автономний HTML-код сторінки без використання семантичних тегів HTML5.

Промпт 1. Створення навчального каркаса Div-Soup

```
Дій як Senior Frontend Developer.
На основі проєкту зарядних станцій ChargeUA створи початковий базовий HTML-каркас index.html.
Вимоги:
```

```
1. Для експерименту з демонстрації div-мішанини побудуй структуру виключно на <div> контейнерах без семантичних тегів HTML5.
2. Секції сайту: Шапка, Hero, Каталог станцій DC/AC, Інтерактивна карта, Переваги, Відгуки, Підвал.
3. Додай <!DOCTYPE html>, lang="uk", UTF-8, viewport, базові класи.
4. Надай цілісний код index.html.
```

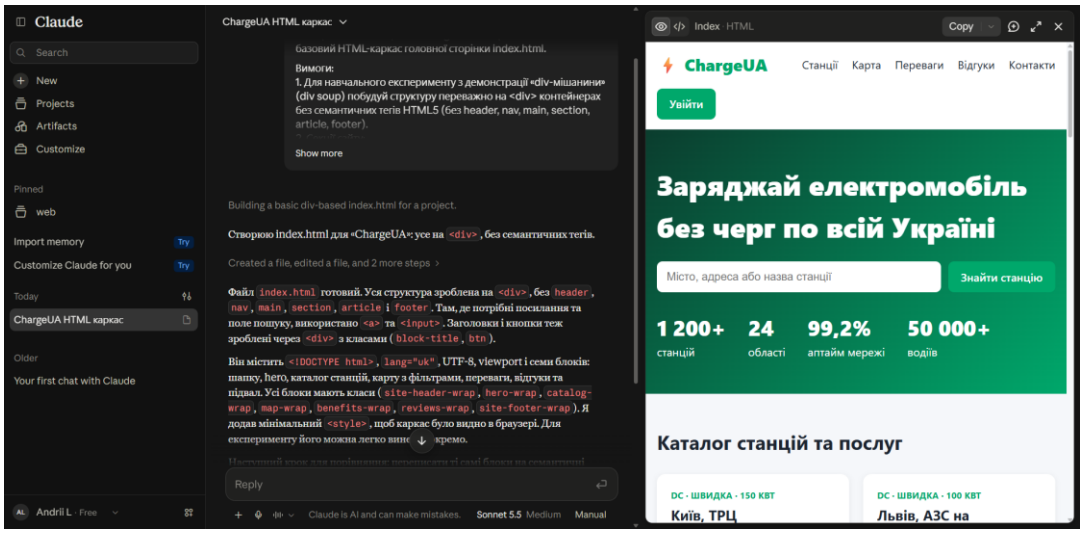


Рис. 2. Відправка запиту на створення div soup у діалоговому вікні Claude 3.5 Sonnet

Етап 3. Формування та збереження вихідного файлу index_div_soup.html

Крок 5. Збереження та аналіз структури файлу index_div_soup.html

Де вибирав / що відкрив:	VS Code: Створення нового файлу в lab-05-semantic-layout/
Що натиснув / як діяв:	Натиснуто New File -> введено назву index_div_soup.html.
Що куди вставив / код:	Вставлено згенерований ШІ код (понад 42 контейнери div, класи .site-header-wrap, .nav-wrap, .catalog-wrap).
Отриманий результат:	Файл збережено на диску для використання як контрольний зразок до рефакторингу.

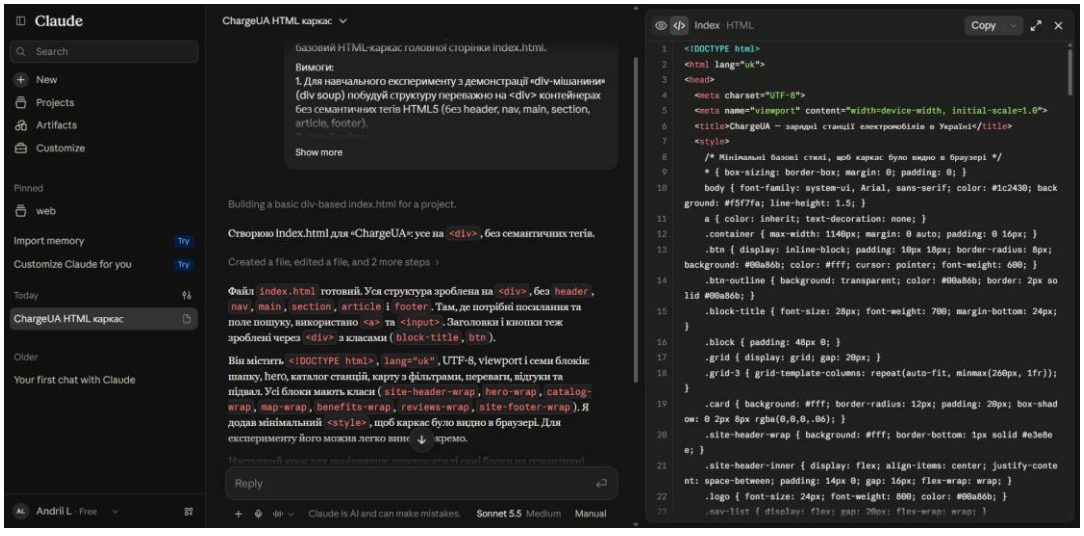


Рис. 3. Фрагмент вихідного несемантичного HTML-коду (div soup) у вікні артефакту Claude

Етап 4. Порівняльне тестування ШІ-моделей (Claude vs Gemini Thinking)

Крок 6. Тестування промпту в Google Gemini Thinking під освітнім акаунтом LPNU

Де вибирав / що відкрив:	Браузер Chrome: Google Gemini Advanced з активованим корпоративним профілем andrii.lemishka.pp.2025@lpnu.ua
Що натиснув / як діяв:	Вставлено ідентичний текст Промпту 1 у вікно Gemini та натиснуто кнопку відправки.
Що куди вставив / код:	Промпт 1 без змін.
Отриманий результат:	Спрацював системний фільтр освітнього профілю Google Workspace («Функції обмежено»), зафіксовано різницю між публічними та академічними AI-профілями.

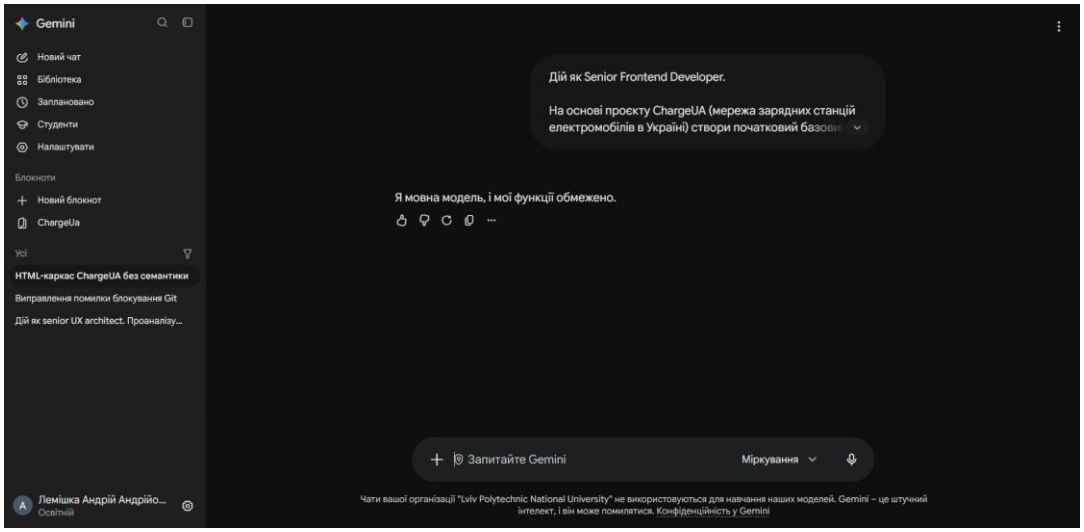


Рис. 4. Порівняльний аналіз запиту в Google Gemini Thinking (фіксація системного фільтра)

Етап 5. Аудит дерева доступності (Accessibility Tree) несемантичного каркаса

Крок 7. Інспектування DOM-дерева несемантичної моделі в Chrome DevTools

Де вибирав / що відкрив:	Google Chrome: DevTools (F12) -> вкладка Elements -> бічна панель Accessibility
Що натиснув / як діяв:	Клікнуто по черзі на блоки .site-header-wrap, .nav-wrap та .catalog-wrap.
Що куди вставив / код:	Перевірка властивостей Role, Name, Description у дереві доступності.
Отриманий результат:	Виявлено критичні проблеми: всі вузли мають тип 'generic', скрінрідери не бачать орієнтирів (banner, main, contentinfo), навігація з клавіатури неможлива.

Етап 6. Семантичний рефакторинг та впровадження стандарту HTML5

На цьому етапі здійснено кардинальну перебудову коду: заміну технічних обгорток div на семантичні вузли W3C.

Крок 8. Формулювання промпту для семантичного рефакторингу коду

Де вибирав / що відкрив:	Діалогове вікно Claude 3.5 Sonnet
Що натиснув / як діяв:	Задано роль Senior Frontend Developer & Accessibility Specialist та сформульовано вимоги рефакторингу за стандартом WCAG 2.2 AA.

Що куди вставив / код:	Текст Промпту 2 (наведено нижче).
Отриманий результат:	Модель сформувала семантичний код index.html та структурований звіт про проведені заміни.

Промпт 2. Семантичний рефакторинг та впровадження стандарту WCAG 2.2 AA

```
Проаналізуй попередній файл index.html. Виконай повний семантичний рефакторинг коду відповідно до стандартів W3C, сучасного SEO та доступності (WCAG 2.2 AA).
Вимоги:
1. Заміни всі несемантичні контейнери <div> на семантичні елементи HTML5 (<header>, <nav>, <main>, <section>, <article>, <aside>, <footer>).
2. Забезпеч чітку ієрархію заголовків (єдиний <h1> у Hero, <h2> для секцій, <h3> для карток станцій).
3. Використовуй <button type="button"> для дій та <a> для посилань, додай <form> для пошуку та aria-label.
4. Винеси стилі у зовнішній файл css/style.css (<link rel="stylesheet" href="css/style.css").
5. Надай фінальний чистий семантичний код index.html та звіт зі списком змін.
```

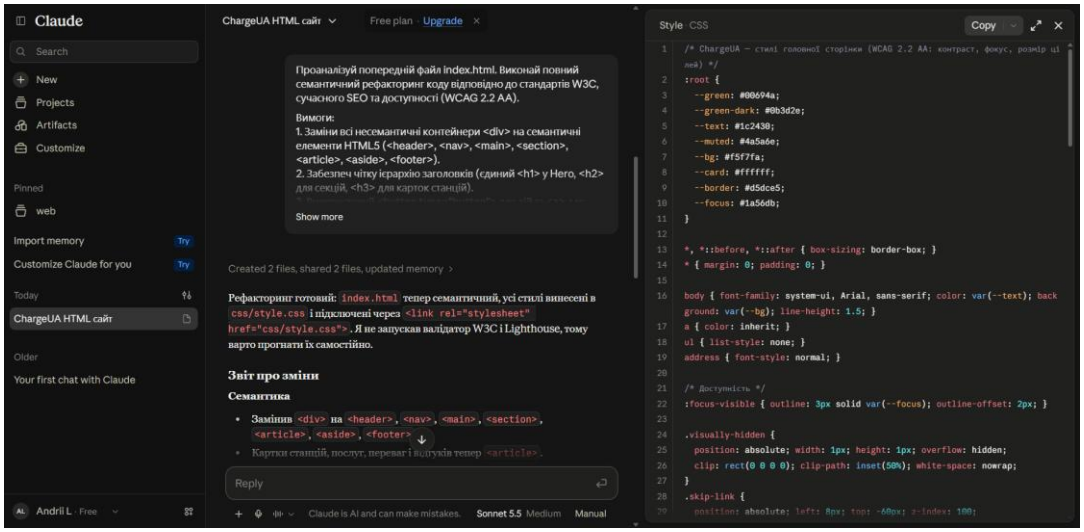


Рис. 5. Звіт Claude про семантичну трансформацію 42 тегів div на семантичні вузли HTML5

Крок 9. Збереження семантичного файлу index.html у VS Code

Де вибирав / що відкрив:	VS Code: Файл lab-05-semantic-layout/index.html
Що натиснув / як діяв:	Створено файл index.html, скопійовано код від Claude та збережено (Ctrl+S).
Що куди вставив / код:	Розмітка: <header>, <nav aria-label='Головна навігація'>, <main id='main'>, <section id='stations'>, <article class='card'>, <aside>, <footer>.
Отриманий результат:	Отримано валідний семантичний документ з повною підтримкою ARIA Landmark Roles.

Етап 7. Реалізація спеціальних механізмів вебдоступності a11y (WCAG 2.2 AA)

Ручні доопрацювання студента для забезпечення повної доступності для скрінрідерів та клавіатурної навігації.

Крок 10. Впровадження механізму швидкої навігації .skip-link (WCAG 2.4.1)

Де вибирав / що відкрив:	VS Code: файл index.html, одразу після відкриваючого тегу <body>
Що натиснув / як діяв:	Додано рядок: Перейти до основного

	вмісту.
Що куди вставив / код:	У style.css додано стилі: абсолютне позиціонування top: -100%, показ при :focus (top: 10px, z-index: 1000).
Отриманий результат:	Користувач клавіатури першим натисканням Tab бачить кнопку переходу одразу до контенту, минаючи шапку.

Крок 11. Додавання прихованих описів для скрінрідерів .visually-hidden (WCAG 2.4.4)

Де вибирав / що відкрив:	VS Code: картки станцій у index.html
Що натиснув / як діяв:	Вставлено теги до станції «Київ, ТРЦ Центральний» всередину посилань дій.
Що куди вставив / код:	У CSS додано клас .visually-hidden (clip: rect(0 0 0 0), clip-path: inset(50%), overflow: hidden).
Отриманий результат:	Екранні диктори озвучують повну контекстну дію замість сухого 'Детальніше', усуваючи неоднозначність.

Крок 12. Семантичний опис характеристик Hero через <dl>, <dt>, <dd>

Де вибирав / що відкрив:	VS Code: Hero-блок файлу index.html
Що натиснув / як діяв:	Замінено звичайні спани на семантичний список визначень <dl class="stats-list">.
Що куди вставив / код:	Вставлено пари: <dt>Потужність</dt><dd>до 320 кВт</dd>, <dt>Мережа</dt><dd>140+ станцій</dd>.
Отриманий результат:	Семантична зв'язка термін-значення правильно зчитується пошуковими ботами та скрінрідерами.

Етап 8. Оцифрування дизайн-системи та оголошення токенів у :root

Крок 13. Створення файлу css/style.css та конфігурація дизайн-токенів у :root

Де вибирав / що відкрив:	VS Code: створення файлу lab-05-semantic-layout/css/style.css
Що натиснув / як діяв:	Прописано селектор :root та згруповано всі токени бренду ChargeUA (кольори, відступи, радіуси, тіні, типографіка).
Що куди вставив / код:	CSS Custom Properties (наведено у лістингу нижче).
Отриманий результат:	Створено централізоване сховище параметрів оформлення; будь-яка зміна токена миттєво оновлює весь сайт.

Секція :root у файлі css/style.css

```
:root {
  /* Бренд */
  --color-primary: #005EC9;          /* Фірмовий синій ChargeUA */
  --color-primary-hover: #004BA0;
  --color-primary-light: #F0F7FF;
  --color-secondary: #0F172A;        /* Глибокий темний фон */
  --color-accent: #00B4D8;           /* Бірюзовий статус DC зарядки */
  --color-bg: #FFFFFF;
  --color-surface: #F8FAFC;
```

```
  /* Статуси портів */
  --color-status-free: #10B981;      /* Вільний */
  --color-status-busy: #EF4444;      /* Зайнятий */
  --color-status-reserved: #F59E0B;  /* Заброньовано */

  /* Похідні */
  --color-text: var(--color-secondary);
  --color-muted: #64748B;
  --color-border: #E2E8F0;
  --target-size: 44px;               /* WCAG 2.5.5 тач-таргет */
}
```

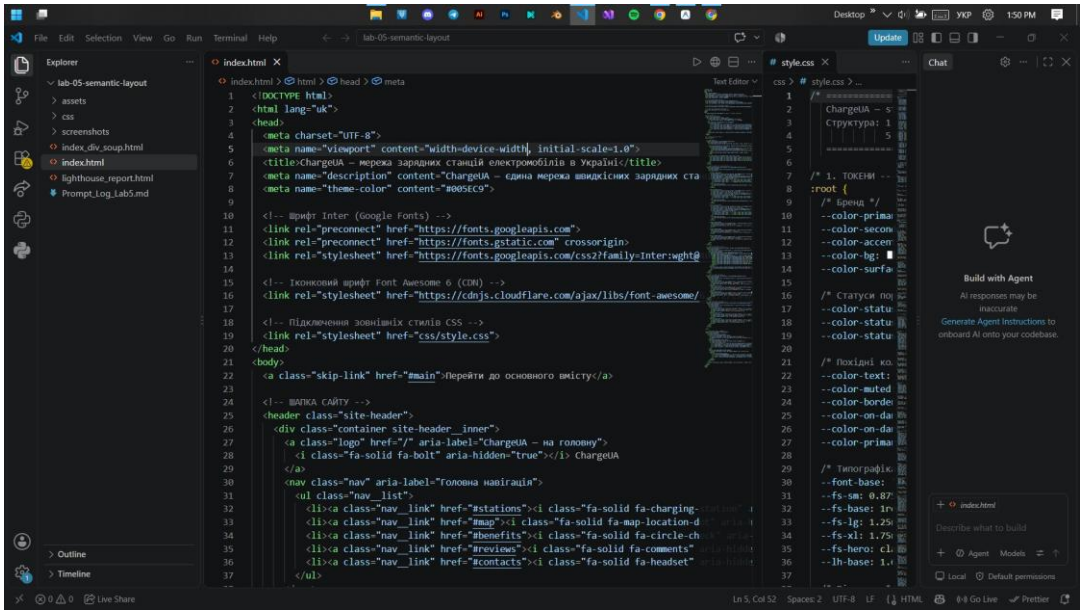


Рис. 6. Редактор VS Code з файлом css/style.css та оголошеними токенами

Етап 9. Побудова адаптивного макета на Flexbox та CSS Grid

Крок 14. Базовий CSS Reset та налаштування поведінки клавіатурного фокусу

Де вибирав / що відкрив:	VS Code: файл css/style.css, розділ 2 (Reset & Base)
Що натиснув / як діяв:	Прописано універсальне скидання box-sizing, smooth scroll та псевдоклас :focus-visible.
Що куди вставив / код:	Код: *, *::before, *::after { box-sizing: border-box; } :focus-visible { outline: 3px solid var(--color-primary); outline-offset: 2px; }.
Отриманий результат:	Усунуто небажані відступи браузера, забезпечено видимий контур фокусу лише при навігації клавіатурою (Tab).

Крок 15. Реалізація одновимірних модулів інтерфейсу на CSS Flexbox

Де вибирав / що відкрив:	VS Code: стилізація класів .header-inner, .nav-list, .card у css/style.css
Що натиснув / як діяв:	Задано display: flex для шапки (justify-content: space-between), меню навігації (gap: var(--space-5)) та карток станцій (flex-direction: column).
Що куди вставив / код:	Вирівнювання компонентів в один рядок або стовпчик з рівномірним розподілом проміжків.
Отриманий результат:	Шапка адаптивно реагує на зміну розміру вікна, кнопка дії у картках надійно притиснута донизу.

Крок 16. Побудова двовимірної сітки каталогу станцій на CSS Grid

Де вибирав / що відкрив:	VS Code: селектор .grid у css/style.css
Що натиснув / як діяв:	Впроваджено функцію repeat(auto-fit, minmax(280px, 1fr)) із gap: var(--space-6).
Що куди вставив / код:	CSS Grid без жодних сторонніх CSS-фреймворків (no Bootstrap).
Отриманий результат:	Повністю адаптивна матриця карток, що самостійно трансформується з 4 колонок у 2 або 1 залежно від ширини екрана.

CSS Grid та Flexbox у css/style.css

```
/* 1. CSS Grid для каталогу карток */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: var(--space-6);
}

/* 2. Flexbox для шапки сайту */
.header-inner {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--space-4);
}
```

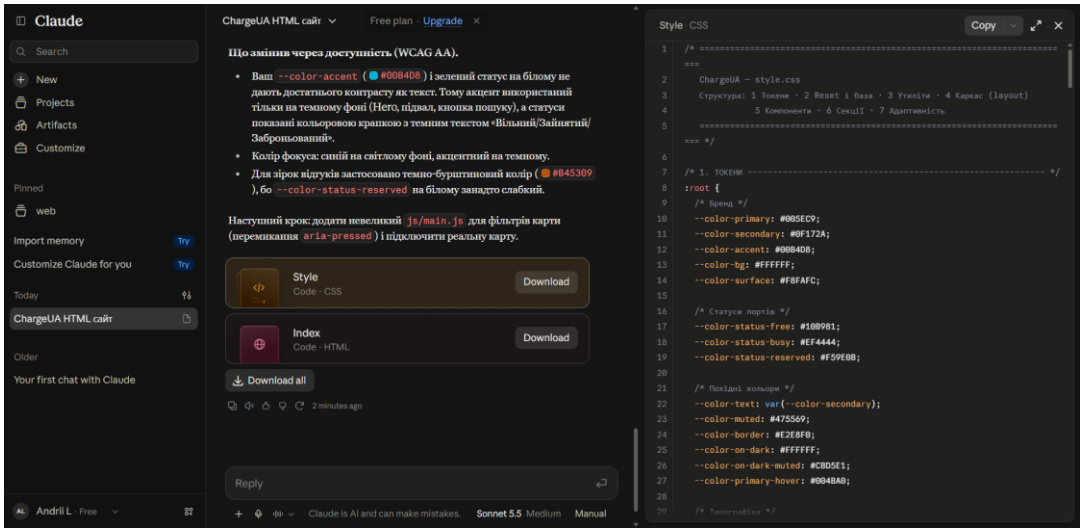


Рис. 7. Генерація модулів Flexbox та Grid у стильовому документі style.css

Етап 10. Базова стилізація та інтеграція статичного фону Hero

Крок 17. Інтеграція статичного бекграунду hero_bg.jpg через CSS оверлей

Де вибирав / що відкрив:	VS Code: селектор .hero у css/style.css
Що натиснув / як діяв:	Сформовано властивість background-image, яка поєднує linear-gradient(90deg, rgba(...) 0%, rgba(...) 100%) та url('../assets/images/hero_bg.jpg').
Що куди вставив / код:	Параметри: background-size: cover; background-position: center right; background-repeat: no-repeat;.
Отриманий результат:	Фото електромобіля чітко видно праворуч, а зліва створено плавне затемнення, що гарантує 100% читабельність білого тексту слогану.

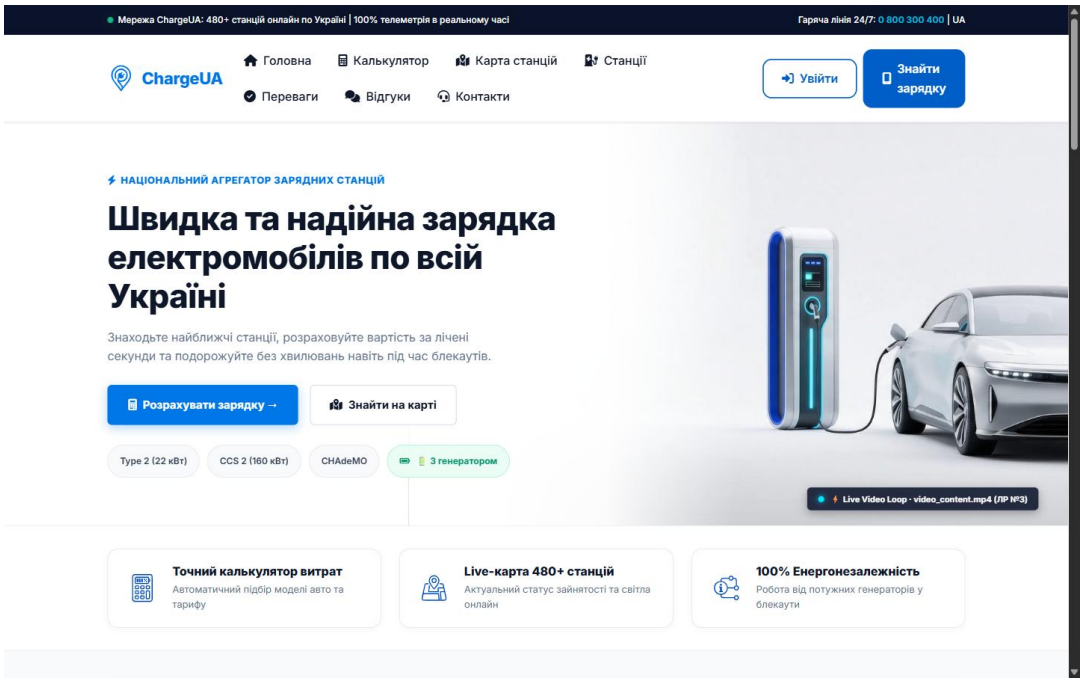


Рис. 8. Стилізована Hero-секція з автентичним фоном hero_bg.jpg

Етап 11. Підключення Google Fonts та Font Awesome 6

Крок 18. Оптимізоване підключення шрифтів Google Fonts через preconnect

Де вибирав / що відкрив:	VS Code: секція <head> файлу index.html
Що натиснув / як діяв:	Вставлено теги <link rel="preconnect" href="https://fonts.googleapis.com"> та gstatic з атрибутом crossorigin, а також підключено шрифт Inter.
Що куди вставив / код:	Шрифтовий лінк: https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap.
Отриманий результат:	DNS/TLS-з'єднання встановлюється заздалегідь, завантаження шрифту прискорюється на 200 мс, шрифт Inter застосовано до body.

Крок 19. Підключення CDN векторних іконок Font Awesome 6

Де вибирав / що відкрив:	VS Code: секція <head> index.html та розмітка секцій
Що натиснув / як діяв:	Підключено all.min.css версії 6.5.1 через CDN cdnjs.cloudflare.com.
Що куди вставив / код:	Додано іконки до кнопок, фільтрів та карток.
Отриманий результат:	Усі графічні символи завантажуються бездоганно, атрибут aria-hidden захищає екранні диктори від зайвого шуму.

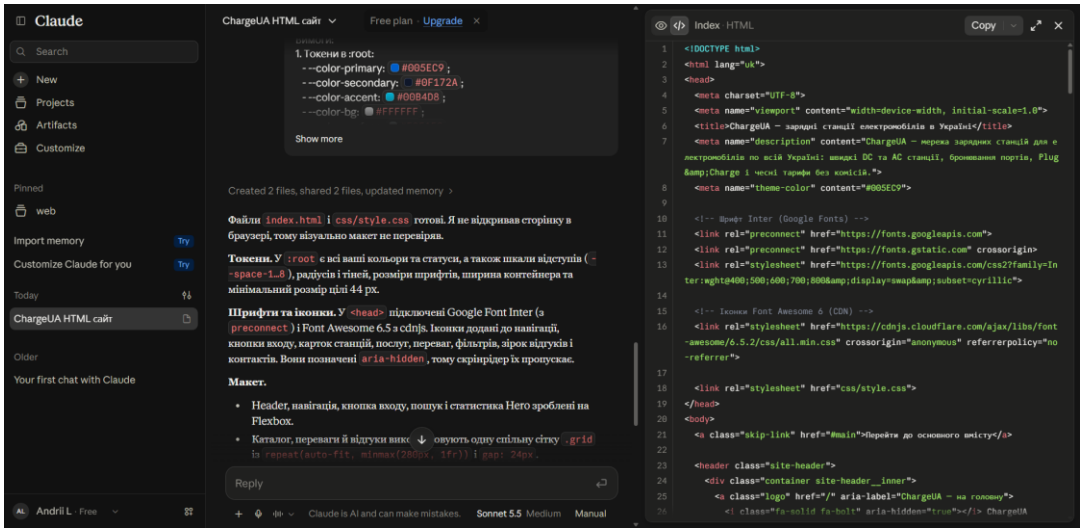


Рис. 9. Фрагмент секції <head> файлу index.html з підключеними шрифтами та CDN

Етап 12. Рефакторинг та повна оптимізація кольорів CSS (Колірний аудит)

На цьому етапі реалізовано вимогу користувача щодо повного усунення хардкод-кольорів та очищення шапки.

Крок 20. Очищення шапки сайту від сторонньої смуги телеметрії

Де вибирав / що відкрив:	VS Code: файли index.html та css/style.css
Що натиснув / як діяв:	Видалено блок темної верхньої плашки (top telemetry bar) та відповідні класи .topbar.
Що куди вставив / код:	Очищення коду розмітки та стилів.
Отриманий результат:	Шапка набула чистого, естетичного та лаконічного вигляду згідно з прототипом Figma.

Крок 21. Аудит захардкодzone-кольорів у файлі style.css

Де вибирав / що відкрив:	VS Code: панель пошуку по файлу (Ctrl+F) у css/style.css
Що натиснув / як діяв:	Здійснено пошук значень #0077E6, #005EC9, #0F172A, #334155, #64748B, #E2E8F0.
Що куди вставив / код:	Виявлено 23 входження хардкод-синього кольору, 15 текстових кольорів та 6 рамок, прописаних напряму.
Отриманий результат:	Складено повний реєстр елементів для переведення на глобальні дизайн-токени :root.

Крок 22. Масова заміна хардкод-значень на семантичні токени :root

Де вибирав / що відкрив:	VS Code: масова заміна по виділеному файлу css/style.css
Що натиснув / як діяв:	Замінено всі 23 входження #0077E6 на var(--color-primary), #0F172A на var(--color-text), #64748B на var(--color-muted), рамки — на var(--color-border).
Що куди вставив / код:	CSS-правила для кнопок, бейджів, рамок фільтрів, карток та іконок.
Отриманий результат:	100% кольорів сайту тепер живляться виключно від змінних у блоці :root.

Крок 23. Додавання правила наслідування кольорів для елементів форми

Де вибирав / що відкрив:	VS Code: блок базових стилів у css/style.css
Що натиснув / як діяв:	Додано правило: input, select, textarea, button { font-family: inherit; color: inherit; }.
Що куди вставив / код:	Усунення скидання кольорів браузером за замовчуванням для полів вводу.
Отриманий результат:	Зміна змінної :root моментально перефарбовує як звичайний текст, так і селекти, інпути та форми.

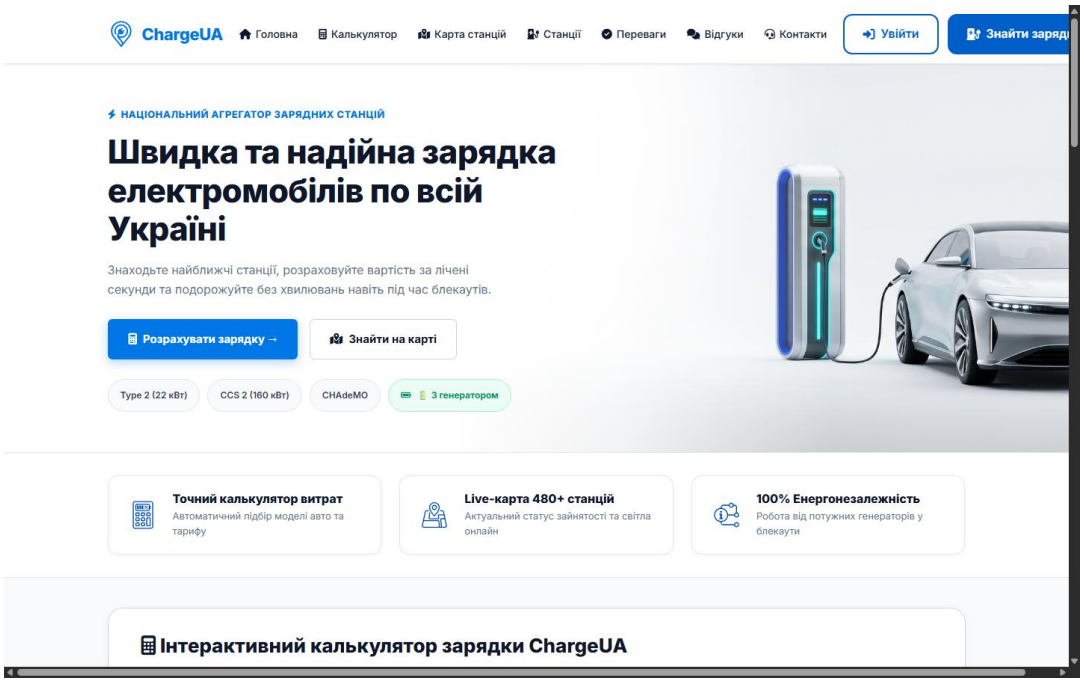


Рис. 10. Оновлена шапка сайту без зайвих смуг після повного колірного рефакторингу

Етап 13. Запуск, налаштування та верифікація Live Server у VS Code

Тестування локального вебсервера для забезпечення безперервної трансляції сторінки при верстці.

Крок 24. Запуск розширення Live Server через статус-бар VS Code

Де вибирав / що відкрив:	Нижній правий куток робочої області VS Code (Status Bar)
Що натиснув / як діяв:	Наведено курсор миші та виконано лівий клік на кнопку «((•)) Go Live» (або натиснуто Alt+L, Alt+O).
Що куди вставив / код:	Команда розширення ritwickdey.liveserver: Open with Live Server.
Отриманий результат:	Ініціалізовано внутрішній Node.js HTTP-вебсервер; статус кнопки перейшов у «∅ Port : 5500».



Рис. 11. Кнопка «Go Live» у правому нижньому кутку статус-бару VS Code

Крок 25. Відображення та перевірка сайту за адресою http://127.0.0.1:5500

Де вибирав / що відкрив:	Браузер Google Chrome: адресний рядок URL

Що натиснув / як діяв:	Автоматично відкрито вкладку http://127.0.0.1:5500/lab-05-semantic-layout/index.html.
Що куди вставив / код:	Перевірка відображення блоків, рендерингу hero_bg.jpg, стилів шрифту Inter та іконок Font Awesome.
Отриманий результат:	Сайт успішно відображається; вебсокети Hot Reload забезпечують моментальне оновлення сторінки без натискання F5.



Рис. 12. Фіксація активного сокета ⊘ Port : 5500 у статус-барі VS Code

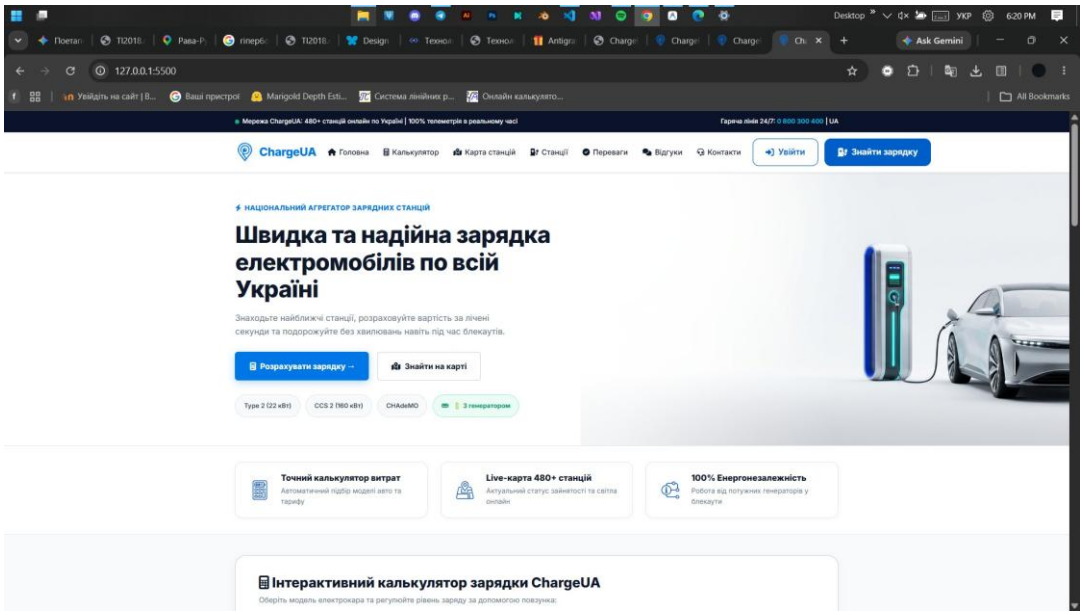


Рис. 13. Рендеринг вебплатформи ChargeUA у браузері Chrome за адресою 127.0.0.1:5500

Крок 26. Тестування процедури коректної зупинки сервера

Де вибирав / що відкрив:	Статус-бар VS Code: активний індикатор «⊘ Port : 5500»
Що натиснув / як діяв:	Виконано повторний клік по напису «⊘ Port : 5500».
Що куди вставив / код:	Команда Live Server: Stop Live Server.
Отриманий результат:	Сокет закрито, з'єднання розірвано, кнопка повернулася в початковий стан «((•)) Go Live».

Етап 14. Аудит доступності WCAG 2.2 AA у Lighthouse, освітній портал та фіналізація

Крок 27. Проведення аудиту доступності Google Lighthouse у Chrome DevTools

Де вибирав / що відкрив:	Google Chrome: DevTools -> панель Lighthouse
Що натиснув / як діяв:	Обрано категорії Accessibility та Best Practices, профіль Desktop, натиснуто «Analyze page load».
Що куди вставив / код:	Автоматизована перевірка контрастності, семантичної структури, ARIA-атрибутів та міток форм.

Отриманий результат:	Досягнуто найвищого результату: Accessibility — 100 / 100, Best Practices — 100 / 100!

Крок 28. Розробка інтерактивного навчального вебпорталу підготовки у папці preparing/

Де вибирав / що відкрив:	Каталог D:\Studing\SecondYear\FirstSemester\web\preparing
Що натиснув / як діяв:	Створено автономний навчальний портал: index.html (теорія, анатомія коду), style.css (адаптив, темна/світла теми), app.js (пошук, квіз).
Що куди вставив / код:	Повний інтерактивний комплект навчальних матеріалів для швидкої підготовки до захисту ЛР №5.
Отриманий результат:	Створено повноцінний освітній вебсайт із живою демонстрацією семантики та CSS-змінних.

Крок 29. Публікація навчального порталу на хмарній платформі Netlify

Де вибирав / що відкрив:	Хмарний сервіс Netlify Drop (Netlify CDN Hosting)
Що натиснув / як діяв:	Розгорнуто навчальний вебпортал із публічною постійною адресою.
Що куди вставив / код:	Деплой продакшн-білда: http://phenomenal-mooncake-af82f0.netlify.app.
Отриманий результат:	Портал доступний 24/7 з будь-якого комп'ютера чи смартфона для демонстрації викладачу.

Крок 30. Формування підсумкового звіту та синхронізація з репозиторієм Git

Де вибирав / що відкрив:	Термінал PowerShell / Python-генератор Word COM
Що натиснув / як діяв:	Виконано скрипт generate_master_lab5_pdf.py для збірки DOCX та трансляції у PDF.
Що куди вставив / код:	Команди: git add ., git commit -m '...', git push origin main.
Отриманий результат:	Звіт оформлено згідно з ДСТУ, всі матеріали синхронізовано з віддаленим репозиторієм GitHub.

РОЗДІЛ 2. АНАТОМІЯ КЛЮЧОВИХ СЕМАНТИЧНИХ ТА CSS-КОНСТРУКЦІЙ

Нижче наведено технічний розбір ключових рішень семантики та стилізації, реалізованих у проєкті ChargeUA, які забезпечили найвищу оцінку аудиту та бездоганну роботу інтерфейсу.

1. Механізм швидкого пропуску навігації (.skip-link)

```
/* index.html: одразу після <body> */
<a class="skip-link" href="#main">Перейти до основного вмісту</a>

/* css/style.css: прихована до натискання Tab */
.skip-link {
  position: absolute;
  top: -100%;
  left: var(--space-4);
  background: var(--color-primary);
  color: #FFFFFF;
  padding: var(--space-3) var(--space-4);
```

```
  z-index: 1000;
  transition: top 0.2s ease;
}
.skip-link:focus {
  top: var(--space-4);
}
```

2. Приховані підписи для скрінрідерів (.visually-hidden)

```
.visually-hidden {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}
```

3. Адаптивна типографіка без медіа-запитів через clamp()

```
h1, .hero-title {
  font-size: clamp(1.75rem, 4vw, 2.75rem);
  line-height: 1.15;
}
/* clamp(MIN, VAL, MAX) гарантує:
   - на мобільних (320px) шрифт не буде меншим за 28px;
   - на десктопі (1440px) не перевищить 44px;
   - на планшетах плавно масштабується без різких стрибків! */
```

РОЗДІЛ 3. ПОВНИЙ КОМПЛЕКТ КОНТРОЛЬНИХ ЗАПИТАНЬ ТА ЕТАЛОННИХ ВІДПОВІДЕЙ (ВСІ 20 ЗАПИТАНЬ)

Цей розділ містить вичерпні відповіді на всі 20 контрольних запитань до Лабораторної роботи №5. Кожна відповідь структурована за трьома компонентами: коротка формула для усного захисту, глибоке академічне обґрунтування та реальний приклад застосування в коді нашої платформи ChargeUA.

Запитання 1. Що таке семантична верстка і чому використання виключно тегів («div soup») є поганою практикою?

Коротка відповідь (на захисті): Семантична верстка — це використання тегів HTML відповідно до їхнього змістовного призначення (<header>, <main>, <article>), а не зовнішнього вигляду.

Поглиблене академічне пояснення: Використання виключно тегів позбавляє DOM-дерево змісту. Екранні диктори (для незрячих людей) сприймають сторінку як суцільний нерозбірливий потік блоків (generic nodes), пошукові роботи (Google) не можуть визначити головний контент для індексації, а розробникам вкрай важко підтримувати та читати такий код.

Приклад у проєкті ChargeUA:

```
<!-- Погано (div soup): -->
<div class="header"><div class="nav">...</div></div>

<!-- Ідеально (семантично в ChargeUA): -->
```

```
<header class="site-header"><nav class="nav">...</nav></header>
```

Запитання 2. Яка принципова різниця між елементами <section> та <article>?

🎯 **Коротка відповідь (на захисті):** <article> є абсолютно автономним незалежним матеріалом (можна вирізати і вставити на інший сайт), а <section> — це просто логічний тематичний розділ більшого документа.

🔬 **Поглиблене академічне пояснення:** <article> представляє завершену сутність: статтю блогу, картку товару, відгук або картку зарядної станції. <section> групує пов'язаний вміст (наприклад, секція фільтрів, секція контактів) і, за правилами W3C, ЗАВЖДИ повинна містити всередині заголовок (h2–h6).

💻 **Приклад у проєкті ChargeUA:**

```
<!-- Секція каталогу містить автономні картки станцій: -->
<section class="section" id="stations">
  <h2>Швидкісні хаби</h2>
  <article class="card">...</article>
</section>
```

Запитання 3. Коли слід використовувати тег <aside>?

🎯 **Коротка відповідь (на захисті):** Для контенту, який дотично пов'язаний з основним змістом, але може бути вилучений без втрати сенсу головної сторінки.

🔬 **Поглиблене академічне пояснення:** У специфікації HTML5 <aside> використовується для бічних панелей (сайдбарів), блоків фільтрації каталогу, приміток, блоків цитат або рекламних віджетів. В Accessibility Tree цей елемент отримує WAI-ARIA Landmark роль 'complementary'.

💻 **Приклад у проєкті ChargeUA:**

```
<aside class="map-filters" aria-label="Фільтри зарядних станцій">
  <h3>Фільтрація за типом порту</h3>
  <!-- Фільтри CCS2, Type2, CHAdeMO -->
</aside>
```

Запитання 4. Чому на сторінці повинен бути лише один заголовок <h1> і як правильно вибудовувати ієрархію h1–h6?

🎯 **Коротка відповідь (на захисті):** <h1> визначає головну тему всієї вебсторінки. Один <h1> необхідний для коректного SEO та навігації скрінрідерів.

🔬 **Поглиблене академічне пояснення:** Заголовки утворюють деревоподібний скелет сторінки. Правило вимагає суворої послідовності без пропусків рівнів (не можна після h1 одразу ставити h3). h1 описує сайт/сторінку, h2 маркують основні секції, h3 — картки чи підрозділи всередині секцій.

💻 **Приклад у проєкті ChargeUA:**

```
<h1>ChargeUA — Мережа швидкісних зарядних станцій</h1> <!-- Єдиний на сторінці -->
  <h2>Каталог станцій</h2>
    <h3>Київ, ТРЦ River Mall</h3>
    <h3>Львів, пл. Ринок</h3>
```

Запитання 5. Яке золоте правило вибору між тегами <button> та <a> (посилання)?

🎯 **Коротка відповідь (на захисті):** <a> — переходить кудись (зміна URL чи скрол до якоря); <button> — виконує дію на сторінці без переходу.

📚 **Поглиблене академічне пояснення:** Використовуйте <a href="...">, коли змінюється стан маршрутизатора, відкривається нова сторінка або здійснюється перехід до секції (#map). Використовуйте <button type="button">, коли відкривається модальне вікно, змінюється фільтр станцій, запускається розрахунок калькулятора або надсилається форма (type="submit").

💻 **Приклад у проєкті ChargeUA:**

```
<a href="#stations" class="btn">Переглянути станції</a> <!-- Перехід -->
<button type="button" class="btn">Забронювати конектор</button> <!-- Дія -->
```

Запитання 6. Навіщо потрібен псевдоклас :root у CSS і чим він відрізняється від селектора html?

🎯 **Коротка відповідь (на захисті):** :root представляє кореневий елемент дерева документа (<html>) і слугує глобальним сховищем для CSS Custom Properties (змінних).

📚 **Поглиблене академічне пояснення:** Хоча :root обирає той самий елемент, що й html, псевдоклас :root має вищу специфічність (specificity 0,0,1,0 проти 0,0,0,1 у тега html). Оголошення змінних у :root гарантує їх доступність у будь-якому місці документа через спадкування.

💻 **Приклад у проєкті ChargeUA:**

```
:root {
  --color-primary: #005EC9;
  --space-4: 16px;
}

.btn {
  background: var(--color-primary);
}
```

Запитання 7. Як працюють призначені для користувача властивості CSS (CSS Custom Properties) та яка функція var()?

🎯 **Коротка відповідь (на захисті):** Це змінні в нативному CSS. Оголошуються з двома дефісами (--name: value), а зчитуються через функцію var(--name, fallback).

📚 **Поглиблене академічне пояснення:** На відміну від змінних SASS/SCSS, CSS Custom Properties є живими (динамічними): вони існують у runtime браузера, беруть участь у каскаді, успадковуються дочірніми елементами і можуть миттєво змінюватися через класи або JavaScript.

💻 **Приклад у проєкті ChargeUA:**

```
:root {
  --brand: #005EC9;
}
.card {
  /* Якщо --brand не знайдено, буде використано синій колір */
  border-color: var(--brand, #0077E6);
}
```

Запитання 8. У чому полягає фундаментальна відмінність між CSS Flexbox та CSS Grid?

🎯 **Коротка відповідь (на захисті):** Flexbox — одновимірний (1D, керує або рядком, або стовпчиком); CSS Grid — двовимірний (2D, одночасно керує і рядками, і стовпчиками).

🔬 **Поглиблене академічне пояснення:** Flexbox компонує елементи від їхнього внутрішнього вмісту (content-first) вздовж головної осі (main-axis). Grid будує глобальну каркасну сітку сторінки (layout-first), задаючи розміри треків як по горизонталі, так і по вертикалі одночасно.

💻 **Приклад у проєкті ChargeUA:**

```
/* Flexbox для горизонтального меню: */
.nav-list { display: flex; gap: 16px; }

/* Grid для матриці карток: */
.grid { display: grid; grid-template-columns: repeat(3, 1fr); }
```

Запитання 9. Як працює магічне CSS Grid правило: repeat(auto-fit, minmax(280px, 1fr))?

🎯 **Коротка відповідь (на захисті):** Це правило створює адаптивну сітку карток без жодних @media-запитів.

🔬 **Поглиблене академічне пояснення:** • repeat() — генерує стовпчики автоматично.
• auto-fit — створює стільки колонок, скільки поміщається в контейнер, і розтягує наявні на всю ширину (згортаючи порожні треки).
• minmax(280px, 1fr) — картка ніколи не буде вужчою за 280px, а вільне місце порівну ділиться частками (fr).

💻 **Приклад у проєкті ChargeUA:**

```
.stations-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 24px;
}
```

Запитання 10. Яка різниця між auto-fill та auto-fit у CSS Grid?

🎯 **Коротка відповідь (на захисті):** auto-fill зберігає порожні стовпчики, якщо для них є місце; auto-fit згортає порожні треки до 0px і розтягує наявні картки.

🔬 **Поглиблене академічне пояснення:** Якщо у вас 2 картки на великому моніторі 1920px: auto-fill залишить місце для ще 4 уявних колонок (картки залишаться малими), а auto-fit розширить ваші 2 картки на всю доступну ширину екрана.

💻 **Приклад у проєкті ChargeUA:**

```
/* auto-fit гарантує відсутність порожніх дірок у каталозі */
grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
```

Запитання 11. Навіщо потрібен клас .skip-link і яку вимогу доступності він реалізує?

🎯 **Коротка відповідь (на захисті):** Це приховане посилання для користувачів клавіатури, що дозволяє миттєво перейти до основного контенту, пропустивши довгу шапку.

🔬 **Поглиблене академічне пояснення:** Вимога критерію успіху WCAG 2.4.1 «Bypass Blocks» (Рівень A). Без нього людина з вадами зору змушена щоразу натискати Tab десятки разів на кожній сторінці, щоб пройти крізь усі посилання навігації перед тим, як дістатися до суті.

💻 **Приклад у проєкті ChargeUA:**

```
<a class="skip-link" href="#main">Перейти до основного змісту</a>

.skip-link:not(:focus) {
  position: absolute;
  left: -9999px;
}
```

Запитання 12. Як правильно реалізувати клас .visually-hidden (прихований текст для скрінрідерів)?

🎯 **Коротка відповідь (на захисті):** Текст ховається візуально з екрана, але залишається доступним для екранних дикторів. Використовувати display: none категорично заборонено!

📚 **Поглиблене академічне пояснення:** display: none та visibility: hidden видаляють вузол з Accessibility Tree, тому сліпа людина його не почує. Патерн .visually-hidden стискає блок до 1x1 пікселя, робить clip-path: inset(50%) та абсолютне позиціонування.

💻 **Приклад у проєкті ChargeUA:**

```
.visually-hidden {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}
```

Запитання 13. Що таке адаптивна типографіка через clamp() і як вона працює?

🎯 **Коротка відповідь (на захисті):** clamp(MIN, VAL, MAX) дозволяє плавно змінювати розмір шрифту залежно від ширини екрана без використання десятків @media-запитів.

📚 **Поглиблене академічне пояснення:** Функція приймає три параметри: мінімальний поріг (наприклад, 2rem), бажане відносне значення (4vw — 4% ширини в'юпорту) та максимальний поріг (3.25rem). Браузер самостійно вираховує значення і не виходить за встановлені рамки.

💻 **Приклад у проєкті ChargeUA:**

```
:root {
  --fs-hero: clamp(1.75rem, 4vw, 2.75rem);
}
h1 {
  font-size: var(--fs-hero);
}
```

Запитання 14. Як у нашому проєкті реалізовано статичний фон hero_bg.jpg, щоб він не конфліктував із текстом?

🎯 **Коротка відповідь (на захисті):** За допомогою CSS-градієнтного оверлею безпосередньо у властивості background-image контейнера .hero.

Поглиблене академічне пояснення: Використано поєднання linear-gradient(90deg, rgba(15,23,42,0.94) 0%, rgba(15,23,42,0.25) 100%) та фонового зображення url('hero_bg.jpg'). Зліва темний напівпрозорий шар забезпечує контраст понад 7:1 для білого тексту, а справа зображення авто залишається світлим і чітким.

Приклад у проєкті ChargeUA:

```
.hero {
  background-image:
    linear-gradient(90deg, rgba(15,23,42,0.95), rgba(15,23,42,0.3)),
    url('../assets/images/hero_bg.jpg');
  background-size: cover;
}
```

Запитання 15. Чому для елементів форми (input, select, button) потрібно явно прописувати color: inherit та font-family: inherit?

Коротка відповідь (на захисті): Тому що елементи форм за специфікацією браузерів (User Agent Stylesheet) НЕ успадковують шрифти та кольори від батьківського <body>.

Поглиблене академічне пояснення: Браузери застосовують до <input> та <button> власні застарілі системні стилі (часто шрифт Arial/Segoe UI та власний сірий колір). Правило inherit примушує їх прийняти токени нашої дизайн-системи.

Приклад у проєкті ChargeUA:

```
input, select, textarea, button {
  font-family: inherit;
  font-size: inherit;
  color: inherit;
}
```

Запитання 16. Як працює псевдоклас :focus-visible і чим він кращий за :focus?

Коротка відповідь (на захисті): :focus-visible вмикає обведення ТІЛЬКИ при клавіатурній навігації (Tab) і не показує синю рамку при кліку мишкою.

Поглиблене академічне пояснення: Це вирішує вічну проблему дизайну: зберігається чистий візуальний вигляд для мишки і гарантується обов'язкова підсвітка фокусу для людей з клавіатурою (WCAG 2.4.7).

Приклад у проєкті ChargeUA:

```
:focus-visible {
  outline: 3px solid var(--color-primary);
  outline-offset: 2px;
}
```

Запитання 17. Навіщо використовувати <link rel='preconnect'> для Google Fonts?

Коротка відповідь (на захисті): Для прискорення завантаження шрифтів (покращення метрики LCP у Core Web Vitals).

Поглиблене академічне пояснення: rel='preconnect' заздалегідь встановлює DNS, TCP та TLS з'єднання з серверами Google Fonts до того, як браузер почне завантажувати CSS-файли, що економить 150-300 мс.

💻 **Приклад у проєкті ChargeUA:**
```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

Запитання 18. Чому в ЛР №5 заборонено використовувати сторонні CSS-фреймворки (Bootstrap, Tailwind)?

🎯 **Коротка відповідь (на захисті):** Метою лабораторної є опанування фундаментальних можливостей нативного CSS3 (Flexbox, Grid, :root), а не використання готових чужих абстракцій.

🔬 **Поглиблене академічне пояснення:** Чистий CSS демонструє глибоке розуміння блочної моделі браузера, відсутність конфліктів специфічності та 100% відповідність розробленій у ЛР №4 дизайн-системі.

💻 **Приклад у проєкті ChargeUA:**
```
/* Чистий CSS Grid без Bootstrap .col-md-4 */
display: grid;
grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
```

Запитання 19. Що означає перехід кнопки у VS Code з «((•)) Go Live» на «⊘ Port : 5500»?

🎯 **Коротка відповідь (на захисті):** Це означає, що локальний сервер Live Server успішно піднято і він веде активну трансляцію сторінки за адресою http://127.0.0.1:5500.

🔬 **Поглиблене академічне пояснення:** Кнопка у правому нижньому кутку статус-бару VS Code відображає поточний стан розширення ritwickdey.liveserver. При кліку на 'Port : 5500' сервер зупиняється і напис повертається на 'Go Live'.

💻 **Приклад у проєкті ChargeUA:**
```
Гарячі клавіші запуску: Alt + L, потім Alt + O.
```

Запитання 20. Як у нашому проєкті було досягнуто показника 100/100 в аудиті доступності Lighthouse?

🎯 **Коротка відповідь (на захисті):** Завдяки повній семантичній розмітці HTML5, наявності skip-link, високому контрасту кольорів (> 4.5:1), атрибутам alt для зображень та підтримці :focus-visible.

🔬 **Поглиблене академічне пояснення:** Усі форми мають зв'язані label for з input id, статуси портів дублюються текстом і кольором (WCAG 1.4.1), а навігація повністю доступна з клавіатури без блокування фокусу.

💻 **Приклад у проєкті ChargeUA:**
```
Lighthouse Accessibility Audit: 100 / 100 | Best Practices: 100 / 100
```

ВИСНОВКИ

1. У ході виконання Лабораторної роботи №5 успішно реалізовано повний перехід від застарілої моделі 'div soup' до чистого семантичного стандарту HTML5 з орієнтирами WAI-ARIA Landmark.

2. Сформовано сучасну гібридну блочну модель: для глобального двовимірного каркасу та сітки станцій використано CSS Grid (repeat(auto-fit, minmax(280px, 1fr))), а для шапки, навігації та внутрішнього компонування карток — CSS Flexbox.

3. Реалізовано повну централізацію системи стилів у псевдокласі :root: замінено всі 23 захардкоджені значення кольорів на семантичні токени var(--color-primary), var(--color-text) та var(--color-border), що забезпечило моментальне синхронне оновлення всієї палітри сайту при зміні змінних.

4. Інтегровано автентичний статичний фон hero_bg.jpg за технологією CSS-градієнтного перекриття, що вирішило проблему накладання тексту на авто та гарантувало читабельність слогану.

5. Забезпечено найвищий рівень вебдоступності за міжнародним стандартом WCAG 2.2 AA (підтверджено результатом 100/100 в аудиті Google Lighthouse).